

Redux y Redux Toolkit

Guia completa para entenderlo desde cero

Tp Ecommerce — Materia Programacion III

Contenido

1. El problema que resuelve Redux
2. Que es Redux
3. Los tres conceptos clave: Store, Action, Reducer
4. Redux Toolkit (RTK) — la version moderna
5. useSelector y useDispatch — como lo usan los componentes
6. Async Thunks — Redux con llamadas a la API
7. Lo que hicimos en el proyecto
8. Diferencias entre useContext y Redux
9. Resumen visual del flujo

1. El problema que resuelve Redux

Imaginate que tienes una app con muchos componentes. El carrito de compras necesita mostrar cuantos productos hay. La barra de navegacion tambien necesita ese numero. El boton "Agregar al carrito" esta en cada producto. Todos necesitan la misma informacion.

Con `useState` y `useContext`, el estado vive dentro de un componente y se "pasa para abajo" por props. Cuando muchos componentes necesitan el mismo estado, empieza el problema:

El problema sin Redux: para que el Navbar sepa cuantos productos hay en el carrito, el estado del carrito tiene que subir hasta el componente padre comun (App), y luego bajar por props hasta el Navbar, el Cart, el ProductCard... Esto se llama "prop drilling" y se vuelve un infierno de mantener.

La solucion con Redux: el estado del carrito vive en UN solo lugar central (el Store). Cualquier componente, desde cualquier parte del arbol, puede leer el estado o modificarlo directamente. Sin pasar props. Sin contextos anidados.

2. Que es Redux

Redux es una libreria para manejar el estado global de una aplicacion. Funciona como una "base de datos en memoria" para el frontend.

La analogia del banco: Pensa en Redux como el banco de tu app. Vos (el componente) no podés ir directo a la caja fuerte a sacar plata. Tienes que llenar un formulario (la Action), darselo al cajero (el Reducer), y el cajero actualiza el saldo (el Store). Nadie puede saltarse el proceso.

Las tres reglas de Redux

Regla 1 — Una sola fuente de verdad: todo el estado de la app vive en un unico objeto (el Store). No hay 10 `useState` dispersos, hay un solo lugar.

Regla 2 — El estado es de solo lectura: no podés modificar el estado directamente. Para cambiarlo, tenes que enviar una Action que describe el cambio.

Regla 3 — Los cambios se hacen con funciones puras: el Reducer es una funcion que recibe el estado actual + la action, y devuelve el nuevo estado. Sin efectos secundarios.

3. Los tres conceptos clave

El Store — la caja fuerte

El Store es el objeto que contiene TODO el estado de la aplicación. Es único (hay uno solo en toda la app) y es la fuente de verdad.

```
// store.js
import { configureStore } from "@reduxjs/toolkit";
import cartReducer from "../cartSlice";

const store = configureStore({
  reducer: {
    cart: cartReducer, // "cart" es el nombre del slice
    favorite: favoriteReducer,
  },
});

export default store;
```

Con `configureStore` le decimos a Redux: "el estado que lleva el nombre `cart` lo maneja `cartReducer`". Así podemos tener múltiples slices organizados.

La Action — el formulario de solicitud

Una Action es un objeto simple que describe QUE quieres hacer. Tiene dos partes: el tipo (`type`) que es el nombre de la acción, y el `payload` que es la información adicional.

```
// Una action de ejemplo (lo que se envía al store)
{
  type: "cart/addToCart",
  payload: { id: 5, name: "Zapatillas", price: 15000, quantity: 1 }
}
```

Ojo: en Redux Toolkit no escribis las acciones a mano. Se generan automáticamente cuando definís los reducers en `createSlice`. Vos solo llamas a la función: `dispatch(addToCart(producto))`.

El Reducer — el cajero que procesa el pedido

El Reducer es una función que recibe el estado actual y la acción, y devuelve el NUEVO estado. Es donde está la lógica de negocio.

```
// Como funciona un reducer (simplificado)
function cartReducer(state = estadoInicial, action) {
  switch (action.type) {
    case "cart/addToCart":
      return { ...state, cartItems: [...state.cartItems, action.payload] };
  }
}
```

```
case "cart/clearCart":  
  return { ...state, cartItems: [] };  
default:  
  return state; // Si no reconoce la action, devuelve el mismo estado  
}  
}
```

4. Redux Toolkit (RTK) — la version moderna

Redux clasico requeria escribir mucho codigo repetitivo: los tipos de las actions, los action creators, los reducers con switch/case... Redux Toolkit (RTK) simplifica todo esto con createSlice.

createSlice es la funcion estrella de RTK. En un solo lugar defines: el nombre del slice, el estado inicial, y todos los reducers. RTK genera automaticamente las actions y el reducer final.

```
// cartSlice.js – todo en un solo lugar
import { createSlice } from "@reduxjs/toolkit";

const cartSlice = createSlice({
  name: "cart", // Nombre del slice

  initialState: { // Estado inicial (como el useState inicial)
    cartItems: [],
    loading: false,
  },

  reducers: { // Las funciones que modifican el estado

    // addToCart: agrega un producto. Si ya existe, suma 1 a quantity.
    addToCart(state, action) {
      const product = action.payload;
      const existing = state.cartItems.find(i => i.id === product.id);
      if (existing) {
        existing.quantity += 1; // RTK permite mutar gracias a Immer
      } else {
        state.cartItems.push({ ...product, quantity: 1 });
      }
    },

    // removeFromCart: elimina un producto por id
    removeFromCart(state, action) {
      state.cartItems = state.cartItems.filter(i => i.id !== action.payload);
    },

    // clearCart: vacia el carrito
    clearCart(state) {
      state.cartItems = [];
    },
  });

// Exportamos las actions (generadas automaticamente por RTK)
```

```
export const { addToCart, removeFromCart, clearCart } = cartSlice.actions;  
  
// Exportamos el reducer para registrarlo en el store  
export default cartSlice.reducer;
```

Immer por debajo: normalmente en JavaScript no puedes mutar un objeto directamente (rompería la inmutabilidad de Redux). RTK usa Immer internamente, que convierte las mutaciones en copias inmutables. Por eso podemos escribir `state.cartItems.push(...)` en lugar de `return { ...state, cartItems: [...state.cartItems, nuevo] }`.

5. useSelector y useDispatch — los hooks de Redux

Para que los componentes React puedan hablar con el Store de Redux, usamos dos hooks de la librería react-redux.

useSelector — para LEER el estado

useSelector recibe una función (el "selector") que extrae la parte del estado que necesitas. El componente se re-renderiza SOLO cuando esa parte cambia.

```
// En Cart.jsx
import { useSelector } from "react-redux";
import { selectCartItems, selectCartTotal } from "../store/cartSlice";

const Cart = () => {
  // Lee cartItems del store. Se actualiza automáticamente cuando cambia.
  const cartItems = useSelector(selectCartItems);
  const total = useSelector(selectCartTotal);
  // ...
};

// Los selectores se definen en el slice:
export const selectCartItems = (state) => state.cart.cartItems;
export const selectCartTotal = (state) =>
  state.cart.cartItems.reduce((sum, item) => sum + item.price * item.quantity, 0);
```

useDispatch — para MODIFICAR el estado

useDispatch te da la función dispatch. Cuando llamas a dispatch con una acción, Redux ejecuta el reducer correspondiente y actualiza el store.

```
// En Cart.jsx
import { useDispatch } from "react-redux";
import { removeFromCart, clearCart } from "../store/cartSlice";

const Cart = () => {
  const dispatch = useDispatch();

  return (
    dispatch(removeFromCart(item.id))>
    Eliminar
  );
};
```


El flujo completo: el usuario hace click → el componente llama a dispatch(action) → Redux ejecuta el reducer → el store se actualiza → useSelector detecta el cambio → el componente se re-renderiza con los nuevos datos.

6. Async Thunks — Redux con llamadas a la API

Redux por defecto es sincrónico: `dispatch` recibe una acción y listo. Pero en un Marketplace real necesitamos hacer llamadas a la API (async). Para eso existe `createAsyncThunk`.

El problema: no podemos hacer `fetch()` dentro de un reducer porque los reducers tienen que ser funciones puras y sincrónicas. Necesitamos un lugar donde hacer la llamada a la API ANTES de actualizar el store.

`createAsyncThunk` — el intermediario async

`createAsyncThunk` crea una "acción asincrónica". Cuando la despachamos, primero hace la llamada a la API, y luego dispara automáticamente tres acciones: `pending` (cargando), `fulfilled` (éxito), `rejected` (error).

```
// cartSlice.js – con async thunk
import { createSlice, createAsyncThunk } from "@reduxjs/toolkit";
import { cartApiService } from "../services/cartApiService";

// 1. Definimos el thunk: hace la llamada a la API
export const fetchCart = createAsyncThunk(
  "cart/fetchCart",
  async ({ userId, token }) => {
    const items = await cartApiService.getCart(userId, token);
    return items; // esto se convierte en action.payload
  }
);

// 2. En el slice, manejamos los tres estados del thunk
const cartSlice = createSlice({
  name: "cart",
  initialState: { items: [], loading: false, error: null },
  reducers: {
    // ... otros reducers ...
  },
  extraReducers: (builder) => {
    builder
      .addCase(fetchCart.pending, (state) => {
        state.loading = true; // Mostramos spinner
      })
      .addCase(fetchCart.fulfilled, (state, action) => {
        state.loading = false;
        state.cartItems = action.payload; // Guardamos los datos
      })
      .addCase(fetchCart.rejected, (state, action) => {
        state.loading = false;
        state.error = action.error.message; // Guardamos el error
      });
  },
});
```

```
}
```

```
// 3. En el componente, despachamos igual que una action normal  
dispatch(fetchCart({ userId: user.id, token }));
```

7. Lo que hicimos en el proyecto

Aplicamos Redux al Marketplace reemplazando los useContext (CartContext y FavoriteContext) por slices de Redux conectados al backend.

Push 1 — El carrito con Redux basico

Instalamos @reduxjs/toolkit y react-redux. Creamos la estructura basica:

Archivo	Que hace
store/store.js	La central de Redux. Registra todos los slices.
store/cartSlice.js	Estado del carrito: cartItems, loading, error. Actions: addToCart, removeFromCart, updateQuantity, clearCart.
App.jsx	Envuelve la app con <Provider store={store}> para que todos los componentes accedan al store.
Cart.jsx	Usa useSelector para leer cartItems, total, itemCount. Usa useDispatch para removeFromCart, updateQuantity, clearCart.
Checkout.jsx	Usa useSelector para leer el carrito. Despacha clearCart al confirmar la compra.
Navbar.jsx	Usa useSelector(selectCartItemCount) para el badge del carrito.
ProductDetail.jsx	Usa useDispatch para despachar addToCartAsync al hacer click en "Agregar al carrito".

Push 2 — Favoritos con Redux + integracion al backend

Agregamos el favoriteSlice y conectamos ambos slices a la API REST del backend:

Archivo	Que hace
store/favoriteSlice.js	Estado de favoritos. Thunks: fetchFavorites, toggleFavoriteAsync. Action local: toggleFavorite.
store/store.js	Se agrega favoriteReducer. El store queda con dos slices: cart y favorite.

<code>services/cartApiService.js</code>	Llamadas HTTP al backend: GET <code>/api/carrito/{id}</code> , POST <code>/agregar</code> , DELETE <code>/vaciar</code> , POST <code>/checkout</code> .
<code>services/favoriteApiService.js</code>	Llamadas HTTP al backend: GET <code>/api/favoritos/{id}</code> , POST <code>/agregar</code> , DELETE <code>/eliminar</code> .
<code>App.jsx</code>	Al loguearse, <code>AppContent</code> dispara <code>fetchCart</code> y <code>fetchFavorites</code> para cargar desde la DB.
<code>ProductCard.jsx</code>	Despacha <code>toggleFavoriteAsync</code> (con backend) o <code>toggleFavorite</code> (local si no hay login).
<code>Favorite.jsx</code>	Lee <code>favoriteItems</code> del store con <code>useSelector</code> . Muestra la grilla de productos favoritos.
<code>Navbar.jsx</code>	Lee tanto <code>selectCartItemCount</code> como <code>selectFavoriteItems</code> para los badges.

Backend Java — endpoints nuevos para Favoritos

Como no existia ningun endpoint de favoritos, los creamos desde cero en Spring Boot:

```
// FavoriteController.java
@RestController
@RequestMapping("/api/favoritos")
public class FavoriteController {

    @GetMapping("/{userId}")
    public List getFavorites(@PathVariable Long userId) { ... }

    @PostMapping("/{userId}/agregar/{productId}")
    public Favorite addFavorite(@PathVariable Long userId,
        @PathVariable Long productId) { ... }

    @DeleteMapping("/{userId}/eliminar/{productId}")
    public void removeFavorite(@PathVariable Long userId,
        @PathVariable Long productId) { ... }
}
```

8. Diferencias entre useContext y Redux

Aspecto	useContext	Redux Toolkit
Proposito	Estado compartido simple	Estado global complejo
Re-renders	Todos los consumidores del contexto	Solo los que usan el dato que cambio
Async	Requiere manejo manual	createAsyncThunk integrado
Devtools	No hay	Redux DevTools para debuggear
Escalabilidad	Provider Hell con muchos contextos	Un solo store bien organizado
Curva de aprendizaje	Baja — solo React	Media — conceptos nuevos
Cuando usarlo	Tema, idioma, auth simple	Carrito, favoritos, datos complejos

Regla practica: usa useContext para cosas simples como el tema oscuro/claro o el usuario logueado. Usa Redux cuando el estado es compartido por muchos componentes, tiene logica compleja, o necesita sincronizarse con un backend.

9. Resumen visual del flujo completo

Asi funciona Redux de punta a punta en nuestro Marketplace:

Paso	Quien actua	Que pasa
1	Usuario	Hace click en "Agregar al carrito" en ProductDetail
2	Componente	Llama a dispatch(addToCartAsync({ product, userId, token })))
3	Async Thunk	Hace POST a /api/carrito/{userId}/agregar/{productId}
4	Backend	Guarda el item en MySQL y devuelve el CartItem creado
5	Thunk fulfilled	Redux ejecuta el extraReducer y agrega el item al cartItems del store
6	Store	El estado global se actualiza con el nuevo producto
7	useSelector	Todos los componentes que leen el carrito se re-renderizan automaticamente
8	Navbar + Cart	El badge del Navbar se actualiza y el Cart muestra el nuevo producto

Glosario rapido

Termino	Definicion simple
Store	La "base de datos" del frontend. Unica fuente de verdad.
Slice	Un pedazo del store. Contiene estado + reducers + actions de un tema.
Action	Un objeto que describe QUE quieres hacer. Como un pedido.
Reducer	La funcion que procesa la action y devuelve el nuevo estado.
dispatch	La funcion para enviar actions al store. Como presionar "enviar".
useSelector	Hook para leer datos del store en un componente.
useDispatch	Hook para obtener la funcion dispatch en un componente.

Async Thunk	Una action asincrona que puede hacer llamadas a la API.
extraReducers	Donde manejamos los estados (pending/fulfilled/rejected) de los thunks.
Provider	El componente de react-redux que conecta el store con la app.
Selector	Funcion que extrae una parte especifica del estado del store.